1. Introduction to Functions

What is a Function?

A **function** is a reusable block of code that performs a specific task. Instead of writing the same code multiple times, we can place it inside a function and call it whenever needed.

Why Use Functions?

Code Reusability
Better Readability
Easy Maintenance
Reduces Duplication
Easier Testing and Debugging

Practical Example

Without Function:

```
num1 = 10
num2 = 20
print("Sum =", num1 + num2)

num3 = 30
num4 = 40
print("Sum =", num3 + num4)
```

With Function:

```
def add_numbers(a, b):
    print("Sum =", a + b)

add_numbers(10, 20)
add_numbers(30, 40)
```

Output

```
Sum = 30
Sum = 70
```

---

2. Function Definition (def Statement)

Syntax

```
def function_name():
    # Function body
    statements
```

- def is the keyword used to define a function.
- Function name should be meaningful.
- Parentheses () are mandatory.
- Indentation is required.

Writing and Calling Functions

Example 1: Simple Function

```
def greet():
    print("Welcome to Python Functions")

greet()
```

Output

```
Welcome to Python Functions
```

Example 2: Function Called Multiple Times

```
def display_message():
    print("Learning Python Functions")

display_message()
display_message()
display_message()
```

Output

```
Learning Python Functions
Learning Python Functions
Learning Python Functions
```

---

3. Function Arguments & Return Values

Arguments allow data to be passed into a function.
Return values allow a function to send data back.

A. Positional Arguments

Arguments are assigned according to their position.

Example

```
def student(name, age):
    print("Name:", name)
    print("Age:", age)

student("Athar", 25)
```

Output

```
Name: Athar
Age: 25
```

Incorrect Order

```
def student(name, age):
    print("Name:", name)
    print("Age:", age)

student(25, "Athar")
```

Output

```
Name: 25
Age: Athar
```

B. Keyword Arguments

Arguments are passed using parameter names.

Example

```
def employee(name, salary):
    print("Name:", name)
    print("Salary:", salary)

employee(salary=50000, name="Ahmed")
```

Output

```
Name: Ahmed
Salary: 50000
```

C. Return Values

A function can return data using the return keyword.

Example
```
def add(a, b):
    return a + b

result = add(10, 20)

print("Result =", result)
```
Output
```
Result = 30
```

D. Returning Multiple Values
Example
```
def calculate(a, b):
    sum_value = a + b
    product = a * b
    return sum_value, product

s, p = calculate(10, 5)

print("Sum =", s)
print("Product =", p)
```
Output
```
Sum = 15
Product = 50
```

Practical Example: Student Result System
```
def calculate_percentage(total_marks, obtained_marks):
    percentage = (obtained_marks / total_marks) * 100
    return percentage

result = calculate_percentage(500, 420)

print("Percentage =", result)
```
Output
```
Percentage = 84.0
```

---

4. Lambda Functions
What is a Lambda Function?
A **lambda function** is an anonymous (nameless) function used for short operations.
Syntax
```
lambda arguments : expression
```
Equivalent Normal Function:
```
def square(x):
    return x * x
```
Lambda Version:
```
lambda x: x * x
```

Example 1: Lambda for Square
```
square = lambda x: x * x

print(square(5))
```
Output

25

Example 2: Lambda for Addition

```
add = lambda a, b: a + b

print(add(10, 20))
```

Output

30